

THINGS TO KNOW:

1. Lab report must contain following sections: (order must be maintained)
 - a) Title /Question
 - b) Theory: The brief overview of the concept /techniques/syntax/idea/tools etc used
 - c) Steps and Commands: The steps performed along with involved commands
 - d) Output: Screenshot of the output
2. Output screen should be captured (use snipping tool), printed and attached in the report.
3. The output screen should reflect the unique identity of each students for that include the following things in commands (to print message in screen) and/or names and/ or paths :
 - Personal Information (Name, Roll No. and Section)
4. Contents should be written on single side of A4 sized paper.
5. Cover page and contents page should be attached in the report appropriately.
6. The works must be submitted within specified deadline.

Content Page Format

Subject: Database Administration

List of Lab Works

Lab No.	Title /Question	Submission Date	Signature
1(a)	This is sample title	2079/03/15	
1(b)	This is another title	2079/03/17	

Lab works / Part-4 / DBA/ BIT-6th semester

First Create PL/SQL stored procedure that displays the information about lab no. and your personal information in required places (CDB and PDB) and execute the stored procedure at beginning or end of the each lab task (for those lab works where SQL Plus is used)

Sample PL/SQL stored procedure:

```
CREATE OR REPLACE PROCEDURE my_information IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Name: Ram Prasad Sharma/Roll No.: 102 / Section: A');
END my_information;
/
```

Execution:

```
SET SERVEROUTPUT ON;
EXEC my_information;
```

14. Exploring Flashback Technology in Oracle.

a) Preparation and Configuration:

Connect as SYS and verify/enable necessary parameters: Check undo_retention and recyclebin via SHOW PARAMETER. Set undo_retention=3600 if lower, and ensure the database is in ARCHIVELOG mode. Provide commands and screenshots of parameter values before/after changes.

b) Flashback Query

Connect as HR, query current salary for employee_id=103 from EMPLOYEES. Update it (e.g., increase by 1000), commit, and note the timestamp. Then, use Flashback Query to retrieve the original salary as of 5 minutes ago. Include commands, screenshots of before/after queries, and the Flashback result.

c) Flashback Version Query

Perform two sequential updates on salary for employee_id=104 (e.g., +500, wait 1 min, +600), committing each. Use Flashback Version Query to view all versions in the last 10 minutes. Explain the VERSIONS_STARTTIME, VERSIONS_OPERATION, and salary columns in your report. Provide commands and screenshots of updates and query output.

d) Flashback Table

Enable row movement on HR.EMPLOYEES. Delete a row (e.g., employee_id=105), commit, and note timestamp. Flashback the table to before the delete using the

- timestamp. Verify the row is restored with a SELECT. Include commands and screenshots of delete, Flashback complete message, and verification query.
- e) Flashback Drop
Drop the HR.LOCATIONS table. Query the recycle bin to confirm. Flashback it to before drop. Verify with a SELECT on a known row (e.g., location_id=1000). Provide commands and screenshots of drop, recycle bin output, Flashback success, and verification.
 - f) Validation and Cleanup
Revert all changes (e.g., update salaries back, re-delete if needed). Attempt a Flashback Query beyond undo_retention to observe ORA-01555 error (snapshot too old), and explain it. Include screenshots of cleanup queries and the error demonstration.

15. Oracle Flashback Version Query and Transaction Query

- a) Preparation and Configuration.
Connect as SYS and grant required privileges to HR: SELECT ANY TRANSACTION and FLASHBACK ANY TABLE. Verify undo_retention via SHOW PARAMETER undo_retention;. Set to 7200 if lower. Provide commands and screenshots of grants and parameter checks.
- b) Flashback Version Query Setup and Execution.
Connect as HR, select current data for employee_id=106 from EMPLOYEES (e.g., salary). Perform three changes: UPDATE salary +1000, commit; wait 1 min, UPDATE +500, commit; wait 1 min, DELETE the row, commit. Then, use Flashback Version Query to retrieve all versions in the last 15 minutes, displaying VERSIONS_STARTTIME, VERSIONS_OPERATION, and salary. Explain the operations (U/D) in your report. Include commands and screenshots of changes and query output.
- c) Analysis of Version Query Results.
From the results, identify the VERSIONS_XID for the delete operation. Discuss how versions help in auditing changes without logs. Perform an INSERT to restore the deleted row using original data from the query. Verify with SELECT. Provide screenshots of XID extraction, restoration, and verification.
- d) Flashback Transaction Query Setup and Execution.
Perform a multi-statement transaction: UPDATE salary for employee_id=107, DELETE for employee_id=108, commit. Note the XID from v\$transaction (query before commit). Then, query FLASHBACK_TRANSACTION_QUERY for that XID, showing OPERATION, TABLE_NAME, and UNDO_SQL. Include commands and screenshots of transaction, XID note, and query results.
- e) Reversing Changes with Transaction Query.
Execute the UNDO_SQL statements from the query to reverse the updates/deletes. Verify the reversal with SELECT queries on the affected rows. Explain potential issues if undo data expires (e.g., ORA-01555). Provide screenshots of UNDO_SQL execution and post-reversal SELECT outputs.
- f) Validation and Advanced Scenario.

Attempt a Version Query beyond undo_retention to simulate snapshot too old error. For Transaction Query, simulate a larger transaction affecting multiple tables (e.g., update EMPLOYEES and DEPARTMENTS in one commit), query it, and undo. Clean up by reverting all changes. Include screenshots of error demonstration and advanced query/undo.

16. Automatic Workload Repository (AWR) in Oracle

a) Preparation and Configuration.

Connect as SYS and verify AWR-related parameters: Query v\$parameter for statistics_level (ensure TYPICAL) and control_management_pack_access (ensure DIAGNOSTIC+TUNING). If not set, adjust them. Also, check default snapshot interval via SELECT extract(day from snap_interval) *24*60+extract(hour from snap_interval)*60+extract(minute from snap_interval) snapshot_interval FROM dba_hist_wr_control;. Provide commands and screenshots of parameter values.

b) Create Baselines with Snapshots.

Create a manual snapshot. Then, connect as HR and generate workload: Run 5-10 queries (e.g., complex JOINS on EMPLOYEES and DEPARTMENTS, some UPDATES with COMMIT). Create another manual snapshot after the workload. List all snapshots via DBA_HIST_SNAPSHOT to confirm the two new ones. Include commands and screenshots of snapshot creations and list output.

c) Generate AWR Report.

Use the awrrpt.sql script to generate an HTML AWR report comparing the two snapshots. Specify the begin and end SNAP_IDs. Provide the full command sequence (including prompts) and a screenshot of the script completion message. Also, include screenshots of the generated report file path and key sections (e.g., Report Summary, Load Profile).

d) Analyze Report Sections.

Open the AWR report and analyze:

- Instance Efficiency Percentages (e.g., Buffer Hit %, Parse CPU to Parse Elapsed %),
- Top Timed Foreground Events (identify any high waits like I/O or CPU),
- SQL Statistics (top SQL by CPU time or executions, note any from your workload).
- Explain potential issues (e.g., high parse time indicates poor application coding) and suggest optimizations (e.g., bind variables).

Include screenshots of these sections and your written analysis.